Product Specs

1. Functional behavior: what the product must do, step by step.
2. User flows: key journeys and system responses.
3. Business/product rules: what triggers, thresholds, permissions, and decisions.
4. AI/agent behavior: inputs, outputs, tools, decision boundaries, and when it must/ must not act.
5. Data requirements: required inputs, fields, freshness, confidence states.
6. Integrations/APIs: which external systems are used and what each provides.
7. UX requirements: only screens/interactions that affect behavior.
8. Acceptance criteria: objective conditions for something to be considered working.
9. Error/edge cases: important failure behavior and fallbacks.
10. Non-goals: what is explicitly out of scope.

1.
TO SHOW
user sets intent → normal spending happens → ambiguous P2P occurs → agent decides whether to ask → Gnani provides actual speech output → agent resolves/stores context → later transaction benefits from that memory → spending drifts → agent decides to intervene → authorised action occurs if applicable → outcome is verified → monitoring resumes

2.
User: Aarav
Date: October 2026
Intent: ₹2,000 dining spending limit
Authority: bounded L3 authority, as defined below

3.

Aarav's Delegated Agent Authority

I authorize my agent to independently monitor my dining expenditure against my monthly spending limit of ₹2,000 for October 2026.

The agent may:

- Monitor and categorize my dining transactions across connected financial accounts and payment methods.
- Maintain my cumulative spending, remaining budget and spending trajectory.
- Ask me brief questions when important transaction context is missing.
- Learn from my confirmed transaction explanations and use that context for future transactions.
- Independently notify me when my spending pace materially exceeds my intended pace.
- Recommend corrective actions and recalculate my remaining spending allowance.
- Apply my previously authorized notification and intervention policies without requesting permission every time.

The agent may not:

- Change, increase or decrease my ₹2,000 spending limit without my explicit approval.
- Expand its own authority or apply policies to other spending categories.
- Make payments, transfer money or move funds independently.
- Block or cancel payments unless I have explicitly authorized a supported payment-control mechanism.
- Treat my decision to knowingly exceed my budget as permission to revise my original spending intention.

My control:

I can override an intervention, knowingly exceed my spending limit, revise my spending intention, withdraw permissions or modify the agent's authority at any time.

The agent must respect my decisions and distinguish between my original intention and my actual spending behaviour.

(
AVAILABLE AUTHORITY GROUPS (Would be understood internally by the agent based on user's answers on specific questions asked while setting intent)

Independent authority
Monitor transactions.
Categorize high-confidence payments.
Maintain spending pace.
Store confirmed context.
Send authorized notifications.

Conditional authority
Ask questions about ambiguous transactions.
Trigger consequential-payment warnings.
Recommend recovery plans.
Apply predefined policies where supported.

Explicit approval required
Change the monthly budget.
Expand permissions.
Make or transfer payments.
Block or cancel payments.
Extend authority to other categories.

)

The agent may independently execute an authorised merchant payment through the supported Pine Labs agent-native payment flow, only within the amount, resource and expiry constraints explicitly approved by the user.

4. OUTCOME
Aarav's spending drift is identified, the agent takes the permitted intervention, the resulting payment/action state is verified, and the agent returns to monitoring with the updated context.

1. PRODUCT OUTCOME

Keep users' actual spending aligned with the pace and limits they set for themselves, without requiring manual tracking.

MoneyMic is not a better expense tracker. It is an agent that continuously understands spending against the user's own financial intention and decides whether anything needs to happen.

2. Problem

Users can usually see where their money went, but that does not necessarily help them change their behaviour while there is still time to do so.

The discovery behind MoneyMic changed the original problem framing. We initially expected users to want better expense tracking, but users described creating their own forms of friction to prevent themselves from drifting. The underlying problem is therefore **the gap between spending intention and actual behaviour**

Current alternatives

- memory;
- UPI/bank histories;
- spreadsheets;
- automatic expense trackers.

3. Target User

Primary user

18 to 30 Y/O, high frequency digital spenders who:

- sets or cares about spending limits;
- frequently makes small digital payments;
- sometimes finds spending running ahead of intention;
- does not want to manually track transactions.

4. Core Product Thesis

Proposed loop:

Transaction happens
- **Understand it**
- **Determine whether context is sufficient**
- **Clarify only when necessary**
- **Evaluate spending pace**
- **Decide what to do**
- **Learn from verified context**
- **Continue monitoring**

agent must make the **smallest useful intervention** that helps the user stay aligned.

5. Product Principles

1. User intention is the reference point

MoneyMic does not decide what the user “should” spend.(because good spending is subjective)

It asks whether actual behaviour is aligned with what the user explicitly intended.

2. Passive by default

No unnecessary notifications, calls or interventions.

3. Context before consequence

If an important transaction is not sufficiently understood, the agent should resolve the uncertainty before taking a consequential action.

4. Voice is selective

Gnani is a capability, not a trigger rule.

MoneyMic decides:**Call / don't call**

based on available context, materiality and existing memory.

5. Memory is evidence, not truth

Verified context should reduce future questioning, but a previous transaction should never become a permanent assumption about every future transaction.

6. Bounded autonomy

MoneyMic can act autonomously only within the authority the user has delegated. It can never expand its own permissions.

6. MVP

The MVP is not defined by no. of features. It is defined by whether we can demonstrate the **core user outcome**.

It must have

User intent

-Define spending intention/limit.

-Define intervention preferences.

-Define what the agent is allowed to do.

Financial monitoring

-Receive supported transaction events.

-Verify and deduplicate transactions.

-Maintain an authoritative transaction ledger.

Context understanding

-Determine what is verified, inferred, unknown or unresolved.

-Do not infer purpose simply from payee/payment type.

Selective clarification

-Determine whether missing context materially matters.

-Ask only the minimum useful question.

-Use voice when appropriate.

-Leave context unresolved when it cannot be reliably established.

Pace evaluation

Calculate deterministically:

-actual verified spend;

-intended pace;

-expected spend to date;

-deviation;

-pace classification.

Agent decision

-stay silent;

-warn;

-intervene within delegated authority;

-request user approval when necessary.

Memory

-Store verified context.

-Reuse it as evidence.

-Allow later evidence to contradict or replace it.

Auditability

Every meaningful decision should be reconstructable.

7. What is NOT being Built Yet

Not in the MVP:

-full investment management;

-universal UPI payment control;

-arbitrary payment blocking/cancellation;

-unrestricted money transfers;

-autonomous actions outside delegated authority;

-physical fulfilment without a legitimate trigger;

-broad financial aggregation across every platform;

-advanced financial coaching.

The partner rails(Gnani, Pine labs, Delhivery) may eventually support these extensions, but they are not required to prove the core hypothesis.

8. Core User Journey

Setup

User defines:

What do I want to stay on track with?
- spending intention
- intervention preference
- agent authority

Normal transaction

Transaction arrives
- verify
- understand
- sufficient context
- calculate pace
- decide
- stay silent / act
- continue monitoring

Ambiguous transaction

Transaction arrives
- verify
- understand
- context insufficient
- agent determines whether clarification is materially useful
- optionally invoke Gnani
- receive user response
- resolve context
- calculate pace
- decide
- store verified context
- continue monitoring

Consequential action

Decision
- verify authority
- execute supported action
- externally verify result
- store outcome
- return to monitoring

9. AI / Agent Requirements

The agent decides

-how to interpret contextual information;

-whether context is sufficient;

-whether clarification is worth the interruption;

-whether to invoke voice;

-which permitted state transition to take;

-whether to warn/intervene/remain silent;

-whether verified context should become memory.

Deterministic system logic decides

-transaction identity/deduplication;

-financial arithmetic;

-verified spending totals;

-state-transition validity;

-permission enforcement;

-external-action verification.

10. Agent Boundaries

MoneyMic operates at L3 bounded autonomy.

Can do independently

-monitor transactions;

-categorise high-confidence transactions;

-maintain spending pace;

-use verified memory;

-send authorized notifications.

Can do conditionally

-clarify ambiguous transactions;

-warn/intervene according to policy;

-recommend corrective actions;

-apply supported policies within delegated limits.

Requires explicit user approval

-change spending intention;

-expand permissions;

-apply authority to a new category;

-transfer money outside explicitly supported authority;

-block/cancel arbitrary payments;

-perform actions outside the delegated boundary.

11. Data Model

For every financial event, MoneyMic should preserve at minimum:

-transaction ID;

-timestamp;

-amount/currency;

-direction;

-merchant/payee;

-payment type/status;

-source;

-raw narration/metadata;

-purpose status;

-category status;

-context status;

-whether it contributes to verified spending.

Confidence states

VERIFIED / INFERRED / UNKNOWN / CONFLICTED / UNRESOLVED

Critical rule:

Unknown is not zero, and inferred is not verified.

12. Partner Rails

The rails are infrastructure. **MoneyMic owns the orchestration and decision logic.**

Pine Labs - Money

Used for:

-consented financial-data access;

-transaction data;

-categorisation/spending intelligence;

-agent authorized payment execution;

-payment-state verification.

MoneyMic's added layer is deciding whether a user's payment should face intervention based on their spending intent.

Gnani - Voice

Used for:

-voice interaction;

-speech recognition;

-conversational handling;

-multilingual interaction.

MoneyMic owns the decision of “when voice is actually necessary”.

Delhivery - Logistics

Used for:

-shipment creation;

-tracking;

-delivery verification;

-address/serviceability.

This is an extension layer, not an MVP dependency.

The capability architecture currently defines 17 capabilities across these three partner rails.

13. UX Requirements

The product should feel like an agent, not another dashboard that demands work.

Onboarding

User should understand:

-what MoneyMic is responsible for;

-what data it needs;

-what it can do;

-what it cannot do.

Dashboard

The primary outcome should be “Being on pace”

Clarification

The interaction should be:

-brief;

-contextual;

-low effort;

-optional when context is already sufficient.

Intervention

Explain:

-what changed;

-why MoneyMic is surfacing it;

-what the user can do next.

Do not moralise about spending.

Returning experience

The product should become quieter with learning, not more intrusive.

14. North Star & Measurement

North Star

Average absolute deviation between cumulative actual spending and cumulative intended spending.

This is deliberately a **product outcome**

Supporting metrics

Track:

-context resolution rate;

-unnecessary-interruption rate;

-intervention effectiveness;

-manual effort avoided;

-decision latency;

-retention.

Guardrails

Watch:

-incorrect classification;

-false positive interventions;

-unnecessary voice interactions;

-unauthorized actions;

-transaction reconciliation failures;

-privacy/security issues;

-cost per active user.

15. Highest Priority Product Experiment

The central question is:

Does selective intervention actually improve spending alignment without making the agent intrusive?

Primary metric

Change in deviation between actual and intended spending.

Guardrails

-interruption rate;

-user corrections;

-trust/permission retention;

-classification errors.

This should be validated before spending significant effort on broader capabilities.

16. Current Open Questions

These are the questions that should stay visible to the team:

1. How complete and fresh does financial data need to be before the “no manual tracking” promise is credible?
2. What level of uncertainty makes an interruption worthwhile?
3. What intervention actually changes behaviour?
4. How much verified history is needed before memory meaningfully reduces interruptions?
5. How much autonomy are users willing to delegate after experiencing the lower-risk version of MoneyMic?

6. Does the initial 18-30 segment show enough sustained need to justify broader investment?